66 DEGREES

.NET Epic Project Architecture and Developer Guide

How the solution is organized, how infrastructure is consumed, and how developers should implement features

PURPOSE Provide one consolidated development standard for every independently deployable Epic project.

Architecture Standard | Version 1.0 | August 2026

Document Purpose and Scope

This guide defines the required architecture for the .NET solution and the development practices that apply to every Epic project. It consolidates the standards in the four uploaded developer guides and adds the common project structure, layer responsibilities, feature-development workflow, testing expectations, and protected configuration rules.

CORE ARCHITECTURAL DECISION Each Epic is implemented as its own .NET project so it can be built, tested, versioned, and deployed independently. Every Epic project follows the same internal folder and dependency structure.

1. Summary of the Uploaded Architecture Documents

The uploaded documents define complementary parts of one end-to-end request path. They should be read as a unified architecture, not as separate implementation options.

Document	What it establishes	Developer takeaway
Domain Controller Architecture Developer Guide	Thin API controllers, route conventions, dependency usage, endpoint patterns, HTTP responses, and synchronous versus asynchronous actions.	Controllers coordinate HTTP traffic and delegate business behavior; they do not own business or database logic.
OracleDAL Developer Consumption Guide	The custom OracleDAL is built on ADO.NET and provides standard Oracle commands, parameter handling, result methods, strongly typed reads, non-query execution, output parameters, bulk insert, lifetime, and disposal. The project does not use Dapper.	Repositories consume the supplied custom OracleDAL NuGet package and never retrieve connection strings or create a competing data-access path.
RepositoryWrapper & Repository Developer Guide	RepositoryWrapper as the central repository entry point, lazy repository creation, shared dependency ownership, and the extension pattern for new repositories.	Add focused domain repositories and expose them through RepositoryWrapper; keep environment and connection configuration centralized.
API IExceptionHandler Developer Guide	Central exception-to-HTTP mapping using NotFoundException, ValidationException, and the global fallback handler with ProblemDetails.	Throw meaningful exceptions and allow the registered handlers to create consistent responses; avoid repetitive controller try/catch blocks.

1.1 Combined Architecture Defined by the Documents

Together, the four documents define the following flow: the API controller receives a request, the service applies business rules, RepositoryWrapper provides the matching repository, the repository executes a SQL constant through the supplied OracleDAL, and centralized exception handling converts failures into standardized API responses.

```
HTTP Request
    → API Controller
        → Service
            → Repository Interface / RepositoryWrapper
                → Domain Repository
                    → Oracle SQL Constant
                        → OracleDAL
                            → Oracle Database

Any unhandled application exception
    → ASP.NET Core exception middleware
        → registered IExceptionHandler
            → standardized ProblemDetails response
```

2. Solution and Deployment Architecture

A .NET project is created for each Epic. The Epic project is the primary unit of ownership and deployment. This prevents unrelated domains from being coupled into one deployable application and allows teams to build and release domain capabilities independently.

Epic project principle	Required outcome
Independent build	The project compiles without requiring unrelated Epic projects to be deployed.
Independent test	The project test suite can run as part of its own pipeline.
Independent deployment	The API and its configuration can be promoted without rebuilding every Epic.
Consistent structure	Developers can move between Epic projects without learning a different folder or dependency model.
Domain ownership	Business logic, entities, repositories, Oracle statements, and tests remain aligned to the Epic domain.

2.1 Standard Epic Project Structure

```
EpicName/
├── Api/
│   ├── Authorization/
│   ├── Controllers/
│   └── Properties/
├── Services/
│   ├── Interfaces/
│   └── Classes/
├── Entities/
│   ├── DTOs/
│   ├── Enums/
│   ├── Interfaces/
│   └── Models/
├── Infrastructure/
│   ├── Interfaces/
│   ├── Repositories/
│   └── Oracle/
└── Test/
    ├── Api/
    ├── Services/
    └── Infrastructure/
```

PLACEMENT CLARIFICATION Services is a top-level folder alongside Api, Entities, Infrastructure, and Test. It contains the project-specific business layer. Its Interfaces and Classes folders should be divided further by business capability when the number of services grows.

3. Folder Responsibilities and Boundaries

3.1 Api

Api is the application boundary. It receives HTTP requests, applies authorization, validates the transport contract, calls the appropriate domain service, and returns the approved HTTP response. API code must remain thin.

Folder	Responsibility	Must not contain
Authorization	Project-specific authorization policies, handlers, requirements, attributes, and supporting classes.	Database access or domain business workflows.
Controllers	Domain controllers and endpoint actions. Controllers bind requests and delegate to services.	SQL, OracleDAL calls, connection logic, or substantial business rules.
Properties	launchSettings.json and other approved project-specific property files.	Secrets or developer changes to protected configuration without architect approval.

3.2 Services

Services contains the project-specific business behavior. It is the primary business-logic layer and contains at minimum Interfaces and Classes. Organize both folders further by capability so corresponding contracts and implementations remain easy to locate.

- Interfaces define the operations the API can request from the business layer.
- Classes implement validation, orchestration, calculations, state transitions, and calls to repositories or approved integrations.
- Services depend on repository interfaces, not OracleDAL and not SQL constants.
- Services throw approved application exceptions when a business outcome must be translated by the global exception pipeline.
- Services should be asynchronous when they perform database or network I/O.

```
Services/
├── Interfaces/
│   ├── Customers/ICustomerService.cs
│   └── Contacts/IContactService.cs
└── Classes/
    ├── Customers/CustomerService.cs
    └── Contacts/ContactService.cs
```

3.3 Entities

Entities contains domain-specific data contracts and supporting types used by the Epic project. Types should be grouped by purpose rather than placed into one large folder.

Entity type	Use
DTOs	Request, response, service, and repository data transfer objects.
Enums	Closed domain value sets used in contracts or business logic.
Interfaces	Entity- or model-level contracts that are not service or repository contracts.
Models	Other domain-specific models and value structures associated with the project and Services layer.

DEPENDENCY RULE Entities must not depend on Api or Infrastructure. Keep DTOs and models free of database connections, controllers, HTTP response construction, and environment configuration.

3.4 Infrastructure

Infrastructure owns the data-access implementation. It translates domain requests into parameterized Oracle operations and maps Oracle results back to domain DTOs or models.

Folder	Responsibility
Interfaces	Repository contracts used by Services classes.
Repositories	Concrete domain repository classes. Each class contains database methods for one focused business area and uses the supplied OracleDAL instance.
Oracle	Oracle SQL statement constants used by specific repository methods. Constants are grouped by domain or repository and contain no user-supplied string concatenation.

TERMINOLOGY NOTE Oracle uses SQL and PL/SQL terminology. The Oracle folder contains Oracle SQL statement constants; “Transact-SQL” or “T-SQL” is SQL Server terminology and should not be used for these Oracle statements.

3.5 Test

Test contains the automated xUnit tests for the Epic project. xUnit is the required testing framework. Organize tests to mirror the production layers and capabilities so a developer can find the corresponding test quickly. Tests are part of the feature implementation, not a follow-up task.

- Use xUnit [Fact] for one fixed test case and [Theory] with data attributes for parameterized test cases.
- Api tests verify endpoint routing, authorization behavior, response codes, serialization, and exception integration.
- Services tests verify business rules, validation, orchestration, and repository interaction through mocks or fakes.
- Infrastructure tests verify parameter construction, SQL selection, mapping, and database behavior at the appropriate test level.

4. Dependency Direction

Layer	May depend on	Must not depend on
Api	Services interfaces; Entities; approved cross-cutting libraries.	Repositories, OracleDAL, SQL constants, or connection strings.
Services	Entities; Infrastructure repository interfaces or an approved abstraction assembly.	Controllers, launch settings, OracleDAL, or concrete database configuration.
Entities	Framework-neutral types and approved shared primitives.	Api, Infrastructure, controllers, configuration, or Oracle client classes.
Infrastructure	Entities; repository interfaces; OracleDAL; Oracle client packages.	Controllers or API response construction.
Test	The layer under test plus approved test libraries.	Production secrets or unauthorized configuration changes.

ARCHITECTURE ENFORCEMENT A controller-to-repository shortcut is not the default development pattern in this consolidated architecture. New features should flow through Services so business logic has a clear home and can be tested independently.

5. Standard Request and Development Flow

1. Inspect the source Oracle function or procedure and confirm its complete return contract: return value, result cursor or result set, output parameters, Oracle data types, nullability, cardinality, status values, and error behavior.
2. Define or update the DTOs, enums, and models in Entities.
3. Define the business operation on a Services interface.
4. Write the service tests first, including success, validation, not-found, and relevant edge cases.
5. Define the repository operation on an Infrastructure repository interface.
6. Add or update the Oracle SQL constant in Infrastructure/Oracle.
7. Implement the repository method using the OracleDAL instance supplied by RepositoryWrapper.
8. Implement the Services-layer business logic and call the repository interface.
9. Add a thin controller action that delegates to the service.
10. Add API/integration and infrastructure tests required by the change.
11. Build the entire Epic project, run all tests, inspect the diff, and only then push code.

6. API Controller Standard

Controllers are responsible for HTTP concerns only. They should be small, predictable, and aligned with the project authorization and exception-handling standards.

- Place controllers in Api/Controllers and authorization artifacts in Api/Authorization.
- Use [ApiController] and the approved route convention.
- Inject the matching service interface through the constructor.

- Use CancellationToken for asynchronous request work when supported by the call chain.
- Return ActionResult<T> or IActionResult according to the approved controller convention.
- Do not instantiate RepositoryWrapper, a repository, or OracleDAL inside a controller.
- Do not add try/catch merely to translate known exceptions into 400, 404, or 500 responses.

```
[ApiController]
[Route("api/[controller]")]
public sealed class CustomerController : ControllerBase
{
    private readonly ICustomerService _customerService;

    public CustomerController(ICustomerService customerService)
    {
        _customerService = customerService;
    }

    [HttpGet("{customerId:long}")]
    public async Task<ActionResult<CustomerDTO>> GetById(
        long customerId,
        CancellationToken cancellationToken)
    {
        CustomerDTO customer = await _customerService.GetByIdAsync(
            customerId,
            cancellationToken);

        return Ok(customer);
    }
}
```

7. Services Layer Standard

The Services layer is the correct location for business validation and orchestration that is being removed from PL/SQL. A service may coordinate multiple repository calls when one business operation requires them, but it should expose one cohesive domain capability to the controller.

```
public interface ICustomerService
{
    Task<CustomerDTO> GetByIdAsync(
        long customerId,
        CancellationToken cancellationToken);
}

public sealed class CustomerService : ICustomerService
{
    private readonly ICustomerRepository _customerRepository;

    public CustomerService(ICustomerRepository customerRepository)
    {
        _customerRepository = customerRepository;
    }

    public async Task<CustomerDTO> GetByIdAsync(
        long customerId,
        CancellationToken cancellationToken)
    {
        if (customerId <= 0)
            throw new ValidationException("Customer ID must be greater than zero.");

        CustomerDTO? customer = await _customerRepository.GetByIdAsync(
            customerId,
            cancellationToken);

        return customer ?? throw new NotFoundException(
            $"Customer {customerId} was not found.");
    }
}
```

8. Repository and RepositoryWrapper Standard

Repository classes contain only domain-specific database operations. RepositoryWrapper remains the controlled composition point that supplies the logger and initialized OracleDAL instance. Repositories must not read appsettings, resolve connection strings, or construct their own DAL.

```
public interface ICustomerRepository
{
    Task<List<CustomerDTO>> GetCustomersAsync(
        long companyId);
}

public sealed class CustomerRepository : ICustomerRepository
{
    #region privates

    private readonly ILogger<CustomerRepository> _logger;
    private readonly OracleDAL _oracleDal;

    #endregion

    #region constructors

    public CustomerRepository(
        ILogger<CustomerRepository> logger,
        OracleDAL oracleDal)
    {
        _logger = logger;
        _oracleDal = oracleDal;
    }

    #endregion

    #region methods

    public async Task<List<CustomerDTO>> GetCustomersAsync(
        long companyId)
    {
        _logger.LogDebug(
            "Retrieving customers for CompanyID {CompanyID}.",
            companyId);

        var parameters = new
        {
            P_COMPANY_ID = companyId
        };

        return await _oracleDal.GetList<CustomerDTO>(
            CustomerSql.GetCustomers,
            parameters);
    }

    #endregion
}
```

8.1 Adding a Repository to RepositoryWrapper

When a new functional area requires database access, create a dedicated repository and register it with RepositoryWrapper. Use the following five-step extension pattern from the uploaded RepositoryWrapper guide.

Step 1 - Create the Repository Class

```
public class PaymentRepository
{
    #region privates

    private readonly ILogger _logger;
    private readonly OracleDAL _oDal;

    #endregion

    #region constructors

    public PaymentRepository(
        ILogger logger,
        OracleDAL oDal)
    {
        _logger = logger;
        _oDal = oDal;
    }

    #endregion

    #region methods

    // Payment-specific methods go here.

    #endregion
}
```

Step 2 - Add Domain Methods

Implement only operations that belong to the repository's domain. Use the supplied OracleDAL instance for Oracle access.

```
public async Task<List<PaymentDTO>> GetPayments(long invoiceId)
{
    var parameters = new
    {
        P_INVOICE_ID = invoiceId
    };

    return await _oDal.GetList<PaymentDTO>(
        PaymentSql.GetPayments,
        parameters);
}
```

Step 3 - Add a Backing Field to RepositoryWrapper

```
private PaymentRepository? _payment;
```

Step 4 - Add the Public Repository Property

```
public PaymentRepository Payment
{
    get
    {
        if (_payment == null)
        {
            _payment = new PaymentRepository(
                _logger,
                oDal);
        }

        return _payment;
    }
}
```

Step 5 - Consume the New Repository

```
List<PaymentDTO> payments =
    await repositoryWrapper.Payment.GetPayments(invoiceId);
```

REPOSITORYWRAPPER PATTERN Use the uploaded guide's five-step pattern exactly: create the focused repository, add its domain methods, add the backing field, add the lazy-loaded public property, and consume it through RepositoryWrapper. Use the actual OracleDAL field name defined by the project.

9. Oracle SQL Constants and OracleDAL Usage

The Oracle folder centralizes statement text so repositories remain readable and database statements are not duplicated. Use one constants class per cohesive repository or business capability. Choose names that describe the operation, not the current caller.

```
public static class CustomerSql
{
    public const string GetCustomers = """
        SELECT
            CUSTOMER_ID AS CustomerID,
            CUSTOMER_NAME AS CustomerName,
            STATUS_CODE AS StatusCode
        FROM CUSTOMER
        WHERE COMPANY_ID = :P_COMPANY_ID
        ORDER BY CUSTOMER_NAME
        """;
}
```

9.1 OracleDAL Rules

CUSTOM ADO.NET NUGET PACKAGE The custom OracleDAL is built on ADO.NET and will be distributed to every Epic project as an approved NuGet package. Developers consume the package through the project dependency; they do not copy its source into an Epic, create a replacement DAL, or change its package version without architect approval.

- Use the OracleDAL instance supplied to the repository; never construct one in feature code.
- Use parameterized operations and explicit OracleDbType values when the contract benefits from explicit typing.
- Remember that OracleDAL binds parameters by name.
- Clear parameters before a second command when reusing the same mutable DAL command state.
- Use the custom OracleDAL, which is built on ADO.NET, for all database access. Dapper is not used in this architecture and must not be added to an Epic project.
- Prefer the custom OracleDAL strongly typed asynchronous read method when result columns map cleanly to DTO properties.
- Use DataTable, DataSet, DataRow, or scalar methods only when the consuming contract actually requires that shape.
- Treat non-query execution failures as exceptions and allow the centralized exception strategy to handle them.

- Do not share a mutable OracleDAL instance concurrently across unrelated requests.

9.2 Source Return Contract Verification

Before converting an Oracle function or procedure, developers must confirm what the source actually returns and preserve that contract correctly in the .NET web service. Do not infer the .NET return type from the procedure name, an existing caller, or a sample result. Review the source signature and implementation, then confirm the behavior with representative executions or an architect or subject-matter expert when the contract is unclear.

Oracle source behavior	Typical .NET representation	Verification required
Function returns one scalar value	The matching primitive, enum, date/time type, decimal, string, or nullable equivalent.	Confirm Oracle type, precision/scale, length, nullability, sentinel values, and conversion rules.
Procedure returns a result cursor or multiple rows	Task<List<T>> or another approved collection contract.	Confirm column names/types, ordering requirements, duplicates, empty-result behavior, and DTO mapping.
Procedure returns zero or one row	Task<T?> when absence is valid, or Task<T> with NotFoundException when absence is exceptional.	Confirm expected cardinality and whether no row means not found, success with no data, or an error.
Procedure exposes multiple OUT parameters	A purpose-specific result DTO containing every required output.	Confirm parameter direction, type, nullability, status/message semantics, and whether outputs remain valid on failure.
Procedure performs a command with no business result	Task, or an approved operation-result type only when the source returns status information.	Do not invent a DTO or return value. Confirm success, row-count, transaction, and exception behavior.
Procedure returns multiple result sets or mixed outputs	An architect-approved composite result model.	Confirm every result set/output, its relationship, ordering, and the API serialization contract before implementation.

RETURN-TYPE RULE The repository method, Services-layer method, controller action, and public API response must agree with the verified Oracle source contract. Tests must prove scalar versus collection behavior, nullability, empty results, precision, dates/times, output parameters, and error/status mapping.

10. Centralized Exception Handling

ASP.NET Core IExceptionHandler implementations own the mapping from application failures to HTTP responses. Developers express the business meaning by throwing an approved exception and allow the exception middleware to handle the response automatically.

Exception/handler outcome	When to use	HTTP result
NotFoundException	A requested domain resource does not exist.	404 Not Found

Exception/handler outcome	When to use	HTTP result
ValidationException	Input or a business rule makes the operation invalid.	400 Bad Request
Global fallback handler	Any unexpected exception not handled earlier.	500 Internal Server Error with a safe message and trace ID

HANDLER ORDER Register specific handlers first and the catch-all fallback handler last. Registration and middleware ordering are architect-owned cross-cutting concerns.

```
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<NotFoundExceptionHandler>();
builder.Services.AddExceptionHandler<ValidationExceptionHandler>();
builder.Services.AddExceptionHandler<GlobalFallbackExceptionHandler>();

var app = builder.Build();
app.UseExceptionHandler();
app.MapControllers();
```

- Do not expose stack traces, SQL text, database details, connection strings, credentials, or internal exception messages to API consumers.
- Use a local try/catch only for a real local responsibility: recovery, retry, cleanup, translation, or adding necessary context while preserving InnerException.
- Do not add or reorder global handlers from feature code. Request architectural review for a new exception category because it changes the API contract.

11. Dependency Injection and Composition

Register controllers, Services-layer classes, repository abstractions, RepositoryWrapper, and database infrastructure in one architect-approved composition location, typically Program.cs through extension methods. Feature developers may add their new service and repository registrations in the designated registration module but must not alter protected infrastructure lifetimes or configuration behavior.

```
builder.Services.AddScoped<ICustomerService, CustomerService>();
builder.Services.AddScoped<ICustomerRepository>(sp =>
    sp.GetRequiredService<RepositoryWrapper>().Customer);
builder.Services.AddScoped<RepositoryWrapper>();
```

LIFETIME RULE Repository infrastructure and its OracleDAL instance must use a request-appropriate scoped lifetime. Do not register mutable OracleDAL or RepositoryWrapper state as a singleton.

12. xUnit Test-Driven Development Standard

xUnit is the required test framework for every Epic project. All methods require appropriate automated xUnit tests. The expected workflow is test-first: define behavior, write a failing xUnit test, implement the smallest correct code, then refactor while keeping the suite green.

XUNIT STANDARD Use xUnit for unit, integration, exception-handler, repository, and regression tests. New test projects and test classes must follow the solution's approved xUnit project configuration, naming, fixtures, and shared test utilities. Do not introduce a second test framework.

Test level	Minimum focus
Services unit tests	Business rules, valid behavior, invalid inputs, not-found behavior, edge cases, and repository interaction.
Controller tests	Routing/action behavior, authorization where applicable, successful responses, and correct delegation.
Exception integration tests	ProblemDetails shape and 400, 404, and 500 mappings through the middleware pipeline.
Repository tests	Correct SQL constant, parameter names/types/directions, verified source return shape, mapping, expected rows/scalars, nullability, and failure behavior.
Regression tests	Every corrected defect receives a test that would have detected it.

12.1 xUnit Service Test Example

```
[Fact]
public async Task GetByIdAsync_WhenCustomerDoesNotExist_ThrowsNotFoundException()
{
    var repository = Substitute.For<ICustomerRepository>();
    repository.GetByIdAsync(42, Arg.Any<CancellationToken>())
        .Returns((CustomerDTO?)null);

    var service = new CustomerService(repository);

    await Assert.ThrowsAsync<NotFoundException>(() =>
        service.GetByIdAsync(42, CancellationToken.None));
}
```

13. Source Control, Build, and Configuration Rules

Rule	Required developer behavior
No compile errors	Never push code to a remote branch when the Epic project or affected solution does not compile.
All tests pass	Run the complete applicable test suite before every push. A failing test blocks the push.
xUnit tests for all methods	Create meaningful xUnit tests for new and changed methods, following TDD and covering failure paths.
Cursor C# rules	Follow the repository-provided Cursor C# rules. They supplement this guide for testing, formatting, and other enforced development practices.
C# regions	Use #region and #endregion to organize class members consistently. Follow the established region names and order for the class type.
Protected connection strings	No developer or other team member may change or alter any connection string, especially a database connection string. Only an architect may make the change, and only after conferring with the other architects.

Rule	Required developer behavior
Protected configuration	Feature developers must not change protected configuration files. Any requested configuration change must be reviewed and directed by an architect.
Escalate configuration needs	If a connection or configuration change appears necessary, stop and raise it to the immediate architect. Do not implement a temporary or local connection-string workaround.

WHY THESE RULES EXIST The framework and rules prevent avoidable rework, configuration drift, broken shared environments, and time lost correcting changes that should not have been made.

13.1 Required C# Region Organization

Developers must use C# regions to keep classes organized and predictable. Group members by responsibility, use the existing project naming convention, and keep the same general order across comparable classes. Do not leave empty regions or place unrelated members in a region merely to shorten the visible file.

```
#region privates

private readonly ILogger<CustomerRepository> _logger;
private readonly OracleDAL _oracleDal;

#endregion

#region constructors

// Constructors

#endregion

#region properties

// Public properties, when applicable

#endregion

#region methods

// Public and private methods, separated further when needed

#endregion
```

- At minimum, use the regions that apply to the class: privates, constructors, properties, and methods.
- Large classes may divide methods further into service methods, public methods, and private methods when that improves navigation.
- Use the same spelling and casing already established in the project. The examples in this guide use lowercase region names.
- Interfaces and very small types may use only the regions that meaningfully apply; do not add empty regions.

14. Definition of Done for a Feature

- ☐ The implementation is located in the correct Epic project and folder.

- ☐ DTOs/models are in Entities and contain no infrastructure concerns.
- ☐ Business logic is in a Services class behind an interface.
- ☐ Database access is behind a repository interface and implemented in a focused repository.
- ☐ Oracle statement text is stored in the matching Infrastructure/Oracle constants class.
- ☐ The repository uses the approved custom OracleDAL NuGet package and parameterized values; Dapper was not added.
- ☐ The source function/procedure return contract was confirmed and is represented correctly by the repository, service, controller, and web-service response types.
- ☐ The controller is thin and calls the matching service.
- ☐ Approved exceptions are thrown according to business meaning; no redundant response-mapping try/catch was added.
- ☐ Unit, integration, and regression tests appropriate to the change exist and pass.
- ☐ All new tests use xUnit and follow the approved xUnit project conventions.
- ☐ The project compiles with zero errors.
- ☐ No connection string was changed or altered. Any architect-made connection-string change was completed only after conferring with the other architects.
- ☐ No protected configuration file was changed without architect direction.
- ☐ C# classes use the required regions and follow the project region naming and ordering convention.
- ☐ The code follows the Cursor C# rules and the applicable uploaded developer guides.
- ☐ The branch diff contains no secrets, debug artifacts, unrelated changes, or commented-out abandoned code.

15. Common Mistakes to Avoid

Mistake	Why it is a problem	Correct approach
Business logic in controllers	It is difficult to reuse and test and mixes HTTP with domain behavior.	Move it to Services.
Controller calls repository directly	It bypasses the defined business layer.	Inject and call the service interface.
Repository creates OracleDAL	It duplicates ownership and can expose configuration.	Use the instance supplied by RepositoryWrapper.
Adding Dapper	It creates an unapproved second data-access approach and bypasses the custom OracleDAL standard.	Use only the custom OracleDAL NuGet package, which is built on ADO.NET.
Guessing the return type	A scalar, row, collection, OUT-parameter set, or empty result can be exposed incorrectly by the web service.	Verify the source signature, implementation, representative results, nullability, and cardinality before coding.
Unorganized class members	Developers lose time locating fields, constructors, properties, and methods, and similar classes become inconsistent.	Use the required C# regions and the established project ordering.
SQL embedded throughout repositories	It becomes duplicated and hard to review.	Store statements in Infrastructure/Oracle constants classes.

Mistake	Why it is a problem	Correct approach
Entity depends on Infrastructure	It reverses dependency direction and couples contracts to Oracle.	Keep Entities framework- and database-neutral.
Controller catches every exception	It duplicates global behavior and produces inconsistent responses.	Throw approved exceptions and use IExceptionHandler.
Changing a connection string	It can break shared environments, expose protected access details, and bypass architecture governance.	Only an architect may make the change after conferring with the other architects.
Pushing with failing tests or compile errors	It blocks others and wastes integration time.	Build and test before pushing; resolve every failure locally.

16. Quick Reference: Where Does This Code Go?

Code artifact	Location
Authorization policy/handler	Api/Authorization
Controller	Api/Controllers
launchSettings.json	Api/Properties
Business service interface	Services/Interfaces/<Capability>
Business service class	Services/Classes/<Capability>
DTO	Entities/DTOs
Enum	Entities/Enums
Other domain model	Entities/Models
Repository interface	Infrastructure/Interfaces
Repository class	Infrastructure/Repositories
Oracle SQL constants	Infrastructure/Oracle
Unit/integration tests	Test/<matching layer>/<Capability>

17. Final Developer Contract

Develop inside the Epic boundary. Keep controllers thin, business logic in Services, data contracts in Entities, and Oracle access in Infrastructure. Consume repositories through the approved abstractions and RepositoryWrapper composition, use the custom OracleDAL NuGet package built on ADO.NET without Dapper, and centralize SQL statements in the Oracle folder. Throw approved exceptions and rely on IExceptionHandler for standardized responses. Write xUnit tests first, keep all tests green, and compile

successfully before pushing. Never change or alter a connection string. Only an architect may make a connection-string change, and only after conferring with the other architects.